

[Nombre del proyecto]
(DAS) Documento Arquitectura de Software
Versión 1.0

Identificación de Documento

Identificación	
Proyecto	“El mirador” – Soluciones MSV
Versión	0.5
Documento mantenido por	Drive - Windows
Fecha de última revisión	11-05-2026
Fecha de próxima revisión	13-04-2026
Documento aprobado por	
Fecha de última aprobación	

Historia de Revisiones

Fecha	Versión	Descripción	Autor
07-04-2026	0.1	Inicio del Informe pequeñas anotaciones	Vicente – Sebastian _ Marcel
08-04-2026	0.2	Escritura del punto 1.6	Vicente – Sebastian _ Marcel
09-04-2026	0.3	Escritura de todo el punto 2	Vicente – Sebastian _ Marcel
10-04-2026	0.4	Correcciones Menores	Vicente
11-05-2026	0.5	Correcciones en base a la retroalimentación	Sebastián
16-05-2026	0.6	Escritura del punto 4.4	Vicente
17-05-2026	0.7	Escritura del punto 4.1.1 a 4.1.4	Marcel
18-05-2026	0.8	Escritura del punto 3 - 4.3.1 - 4.3.3 y 4.5	Vicente
19-05-2026	0.9	Correcciones menores	Vicente
19-05-2026	1.0	Cambio de requerimientos	Sebastián
20-05-2026	1.1	Escritura del punto 4.1.6 y correcciones menores	Marcel - Vicente

Tabla de Contenidos

1. INTRODUCCIÓN	4
1.1. CONTEXTO DEL PROBLEMA	4
1.2. PROPÓSITO	4
1.3. ÁMBITO	4
1.4. DEFINICIONES, ACRÓNIMOS Y ABREVIACIONES	4
1.5. RESUMEN EJECUTIVO	4
1.6. ARQUITECTURA DEL SISTEMA	4
2. VISIÓN DEL SISTEMA	4
2.1. DESCRIPCIÓN GENERAL DEL SISTEMA	4
2.2. OBJETIVOS DEL SISTEMA	4
2.3. REQUERIMIENTOS FUNCIONALES	4
2.4. REQUERIMIENTOS NO FUNCIONALES	5
2.5. SUPUESTOS Y DEPENDENCIAS	5
3. ESTILOS Y PATRONES ARQUITECTÓNICOS	5
3.2. JUSTIFICACIÓN DEL ESTILO SEGÚN EL CONTEXTO DEL SISTEMA	5
4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS	5
4.1. VISTA DE ESCENARIO	5
4.1.1. <i>Propósito</i>	5
4.1.2. <i>Actores</i>	5
4.1.3. <i>Diagrama general de casos de uso</i>	5
4.1.4. <i>Diagrama de casos de uso específicos</i>	5
4.1.6. <i>Especificación de casos de uso</i>	5
4.2. VISTA LÓGICA	6
4.2.1. <i>Propósito</i>	6
4.2.2. <i>Diagrama de clases</i>	6
4.2.3. <i>Descripción diagrama de clases</i>	6
4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO	7
4.3.1. <i>Propósito</i>	7
4.3.2. <i>Diagrama de componente</i>	7
4.3.3. <i>Descripción diagrama de componente</i>	7
4.3.4. <i>Diagrama de paquete</i>	7
4.3.5. <i>Descripción diagrama de paquete</i>	7
4.4. VISTA DE PROCESOS	7
4.4.1. <i>PROPÓSITO</i>	7
4.4.2. <i>DIAGRAMA DE ACTIVIDAD</i>	7
4.4.3. <i>DESCRIPCIÓN DIAGRAMA DE ACTIVIDAD</i>	7
4.5. VISTA FÍSICA	7
4.5.1. <i>Propósito</i>	7
4.5.2. <i>Diagrama de despliegue</i>	7
4.5.3. <i>Descripción diagrama de despliegue</i>	7
5. REQUISITOS DE CALIDAD	7

5.1. PROPÓSITO	7
5.3. <i>Reglas y criterios de evaluación de calidad</i>	7
6. PRINCIPIOS DE DISEÑO APLICADOS	8
6.1. <i>Propósito</i>	8
6.2. PRINCIPIOS DE DISEÑO (POR EJEMPLO: ABSTRACCIÓN, ACOPLAMIENTO, COHESIÓN, ENCAPSULAMIENTO, MODULARIDAD)	8
7. PROTOTIPO	8
7.1. PROPÓSITO	8
7.2. MOCKUPS (IMÁGENES CON UNA BREVE DESCRIPCIÓN)	8
7.3. JUSTIFICAR HERRAMIENTAS DE PROTOTIPADO	8
8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN	8
8.1. PROPÓSITO	8
8.2. LISTA DE VERIFICACIÓN	8
8.3. ANÁLISIS Y MÉTRICAS DE RESULTADOS	8
9. CONTROL DE VERSIONES	8
9.1. PROPÓSITO	8
9.2. CONTROL DE VERSIÓN UTILIZADO (JUSTIFICAR EL TIPO DE CONTROL DE VERSIÓN UTILIZAD (FECHA, SEMÁNTICA O SECUENCIAL)	8
9.3. JUSTIFICAR HERRAMIENTAS DE VERSIONAMIENTO	8
7. CONCLUSIONES	8
8. BIBLIOGRAFÍA	8
9. ANEXOS	8
9.1. CARTA GANTT	8

1. INTRODUCCIÓN

1.1. Contexto del Problema

El edificio "El Mirador", con 160 departamentos, presenta una crisis operativa por procesos manuales, falta de transparencia y una morosidad crítica. Actualmente, la administración carece de un marco de trabajo técnico y de gestión que permita automatizar el recaudo y la comunicación. Esta propuesta busca implementar una solución basada en ingeniería de software para restaurar la confianza y la liquidez operativa.

1.2. Propósito

El propósito de este documento es definir las especificaciones de software para la creación de una solución tecnológica que automatice la gestión de gastos comunes del edificio "El Mirador". El sistema busca erradicar la ineficiencia de los procesos manuales, reducir la morosidad y asegurar que la administración cuente con liquidez para las mantenciones esenciales. El documento está dirigido al equipo de desarrollo y a la administración del edificio para garantizar la transparencia.

1.3. Ámbito

Gestión de Residentes: Registro digital de datos de contacto y roles (propietario/arrendatario) para centralizar la comunicación.

Gestión de Pagos y Morosidad: Automatización del ciclo de cobro de Gastos Comunes (GC) y Extraordinarios (GE), incluyendo la generación de recibos y el envío de recordatorios de deuda.

Operaciones y Mantención: Módulo para la trazabilidad de solicitudes de reparación y quejas, permitiendo al personal de mantenimiento gestionar sus tareas de forma digital.

Exclusiones: El sistema no procesa dinero directamente (redirección bancaria) ni reemplaza las funciones de seguridad física de la conserjería

1.4. Definiciones, acrónimos y abreviaciones

CVDS: Ciclo de Vida del Desarrollo de Software.

Metodología Incremental: Estrategia de desarrollo donde el sistema se construye y entrega por módulos funcionales.

Stakeholders: Interesados del proyecto (Administración, Residentes, Personal).

GC / GE: Gastos Comunes / Gastos Extraordinarios.

1.5. Resumen ejecutivo

El presente documento detalla los requerimientos para el desarrollo de "Soluciones

MSV" una solución tecnológica diseñada para mitigar las deficiencias administrativas del edificio "El Mirador". Actualmente, la gestión manual de gastos comunes para 160 departamentos ha derivado en una alta morosidad, falta de liquidez y desconfianza en la comunidad. El sistema propuesto automatizará el ciclo de facturación, optimizará la comunicación entre administración y residentes (propietarios y arrendatarios), y garantizará la transparencia financiera mediante reportes en tiempo real. Con la implementación de Soluciones MSV, se espera una reducción significativa de la morosidad y una mejora en la seguridad operativa del edificio al asegurar fondos para mantenciones críticas.

1.6. Arquitectura del sistema

Vista de Escenarios (Casos de Uso)

Esta vista define el comportamiento del sistema desde la perspectiva de los usuarios finales.

- **Administrador:** Es el actor principal que utiliza el sistema para gestionar la cobranza, generar reportes de morosidad, gestionar el pago a funcionarios del edificio y supervisar los contactos con proveedores para la compra de insumos.
- **Comité Junta de Vecinos:** Este grupo de residentes interactúa con el sistema para revisar información administrativa, evaluar reportes y respaldar las decisiones de gestión del edificio.
- **Residentes (Propietario/Arrendatario):** Interactúan con el sistema para consultar su historial, realizar el pago de Gastos Comunes y Extraordinarios, generar solicitudes de mantenimiento o quejas, y recibir notificaciones de llegada de paquetería.
- **Personal del Edificio:** Este actor agrupa a los conserjes, que acceden para registrar visitas y notificar la llegada de paquetería, y al personal de mantenimiento, que interactúa para revisar solicitudes de reparación y gestionar compras de insumos para sus labores.

Vista Lógica

Describe la organización interna del software en módulos funcionales que dan soporte a los requerimientos del negocio.

- **Módulo de Usuarios:** Encargado del registro, actualización y desactivación de usuarios, además de la gestión y asignación de roles y permisos diferenciados según cada perfil.
- **Módulo de Residentes:** Administra la información de los propietarios y arrendatarios manteniendo su trazabilidad histórica, junto con el registro y control de las unidades habitacionales.
- **Módulo Financiero y Cobranza:** Centraliza el cálculo automático de cuotas, el registro de pagos de gastos comunes y extraordinarios, la generación de comprobantes y la gestión automática del estado de morosidad.

- **Módulo de Comunicaciones:** Responsable de la configuración, programación y envío de notificaciones automáticas a múltiples usuarios, conservando además un historial de todas las comunicaciones enviadas. **Módulo de Solicitudes:** Gestiona el ciclo completo, desde el registro y la actualización hasta el cierre, de las solicitudes de mantenimiento y los reclamos o quejas ingresadas al sistema.
- **Módulo de Reportes y Análisis:** Permite la generación, exportación y programación automática de informes financieros, de morosidad, pagos y estados de cuenta individuales para apoyar el control y la toma de decisiones.

Vista de Desarrollo (Implementación)

Define la estructura del software desde la perspectiva de la ingeniería y la programación.

- **Patrón de Arquitectura:** Se utilizará un patrón Modelo-Vista-Controlador (MVC) apoyado en una arquitectura Cliente-Servidor.
- **Frontend (Capa de Presentación):** Interfaz de usuario web responsiva, accesible tanto desde computadores de escritorio (para la oficina de administración y conserjería) como desde dispositivos móviles (para la comodidad de los residentes).
- **Backend (Capa de Lógica de Negocio):** Una API (Interfaz de Programación de Aplicaciones) que procesará las reglas de negocio, como el cálculo automático de multas por morosidad.
- **Base de Datos:** Se empleará un motor de base de datos relacional para garantizar la integridad transaccional, dado que el sistema manejará información financiera crítica e historial de pagos.

Vista de Proceso

Describe el comportamiento dinámico del sistema, el flujo de la información y el manejo de la concurrencia:

- **Concurrencia de Pagos:** El sistema estará preparado para manejar múltiples sesiones simultáneas, especialmente durante los primeros días del mes, cuando el volumen de residentes consultando y registrando pagos aumenta significativamente.
- **Sistema de Notificaciones:** Procesos automáticos encargados de disparar alertas (ej. correos electrónicos o notificaciones push) cuando un conserje registra un paquete o cuando se acerca la fecha de vencimiento de un gasto común.

Vista Física (Despliegue)

Muestra cómo se distribuirá el software en el hardware y la infraestructura tecnológica:

- **Infraestructura Cloud:** El sistema (Servidor de Aplicaciones y Servidor de Base de Datos) estará alojado en la nube (Cloud Computing) para garantizar

una disponibilidad del 24/7 y facilitar las copias de seguridad automáticas sin requerir servidores físicos dentro del edificio.

- **Nodos de Acceso:** * **Conserjería y Administración:** Computadores de escritorio conectados a la red mediante navegadores web estándar.
 - **Residentes y Personal de Mantención:** Acceso a través de navegadores web en smartphones o computadores personales desde cualquier ubicación con conexión a internet.

2. VISIÓN DEL SISTEMA

2.1. Descripción general del sistema

"Soluciones MSV" es una plataforma integral para centralizar la gestión financiera y operativa de "El Mirador". Para su ejecución, se adoptará una **Metodología de Desarrollo Incremental** apoyada en un enfoque de gestión **Ágil**. Esto permitirá entregar primero el módulo financiero (crítico para la morosidad) y, en iteraciones posteriores, los módulos de conserjería y mantenimiento, garantizando que cada entrega aporte valor inmediato al cliente.

A grandes rasgos, el software gestionará cuatro áreas clave:

- **Área Financiera:** Automatización del cobro y registro de gastos comunes, extraordinarios, morosidad y multas, además de facilitar la gestión del pago de sueldos a los funcionarios.
- **Comunidad:** Mantenimiento de un directorio digital actualizado con la información de todos los habitantes y la asignación de cada departamento.
- **Mantención y Operaciones:** Canalización de solicitudes de reparación y quejas por parte de los residentes, junto con un módulo para gestionar proveedores y compra de insumos.
- **Conserjería:** Registro de correspondencia y paquetería, con envío de notificaciones automáticas a los residentes.

2.2. Objetivos del sistema

Desarrollar un sistema centralizado que automatice la gestión financiera y operativa del edificio "El Mirador", con el fin de reducir la morosidad, transparentar los recursos y optimizar la comunicación entre la administración, el personal y los residentes.

Objetivos Específicos:

1. **Automatizar el cobro** de gastos comunes, extraordinarios y multas para disminuir la morosidad.
2. **Centralizar el directorio** digital de propietarios y arrendatarios con la asignación de sus respectivos departamentos.

3. **Agilizar los procesos administrativos**, como el pago de remuneraciones a funcionarios y la compra de insumos a proveedores.
4. **Controlar la paquetería** en conserjería, enviando notificaciones automáticas a los residentes al recibir correspondencia.
5. **Canalizar solicitudes y quejas** de los residentes mediante un módulo digital, ordenado y trazable.
6. **Generar reportes** financieros y de gestión en tiempo real para la toma de decisiones.

2.3. Requerimientos funcionales

[R-N°]	[Nombre del Requerimiento]	Descripción Detallada (Acción Técnica)
RF01	Gestión de usuarios	Permite el registro, actualización y desactivación de usuarios (identificación, contacto, rol, medio de notificación y estado). No permite eliminación física.
RF02	Gestión de roles y permisos	Permite la asignación y actualización de roles y permisos diferenciados según el tipo de usuario.
RF03	Gestión de usuarios y perfiles	Permite el registro, actualización y desactivación de perfiles de usuario para asegurar el control de acceso.
RF04	Gestión de residentes	Permite el registro, actualización y desactivación de residentes, manteniendo la trazabilidad histórica.
RF05	Gestión de unidades habitacionales	Permite el registro, actualización y desactivación de departamentos.
RF06	Cálculo automático de cuotas	Permite el registro/actualización de parámetros de cálculo y la generación automática de las cuotas de gastos comunes.
RF07	Gestión de gastos comunes y pagos	Permite el registro, actualización y desactivación (anulación lógica) de pagos de gastos comunes.

RF08	Gestión de gastos extraordinarios	Permite el registro, actualización y desactivación de gastos extraordinarios.
RF09	Generación de recibos de pago	Permite la generación y reimpresión de comprobantes de pago.
RF10	Gestión de morosidad	Permite el registro automático, actualización y desactivación automática del estado de morosidad de las unidades.
RF11	Programación de notificaciones	Permite el registro, actualización y activación/desactivación de notificaciones asociadas a eventos específicos.
RF12	Programación de notificaciones	Permite el registro, actualización y activación/desactivación de notificaciones asociadas a eventos específicos.
RF13	Notificaciones múltiples	Permite el envío de notificaciones a múltiples usuarios relacionados con una misma propiedad.
RF14	Historial de comunicaciones	Permite el registro automático, consulta y conservación del historial de todas las notificaciones enviadas.
RF15	Registro de solicitudes de mantenimiento	Permite el registro, actualización y cierre (desactivación) de solicitudes de mantenimiento.
RF16	Gestión de solicitudes de quejas	Permite el registro, actualización y cierre (desactivación) de reclamos.
RF17	Acceso a información en tiempo real	Permite la consulta en línea de información relevante según el perfil de usuario.
RF18	Consulta de información por parte de residentes	Permite a los residentes la visualización de su estado de cuenta, pagos y deudas.
RF19	Consulta administrativa	Permite la configuración y

	avanzada	actualización de criterios de búsqueda para generar consultas dinámicas.
RF20	Búsqueda y filtros	Permite la configuración y el uso de filtros avanzados de búsqueda en el sistema.
RF21	Panel de control administrativo	Permite la visualización de indicadores clave de gestión (Dashboard administrativo).
RF22	Generación de informes generales	Permite la generación de informes con configuración y actualización de parámetros.
RF23	Generación de reportes financieros	Permite la generación y configuración de reportes financieros del edificio.
RF24	Reporte de morosidad	Permite la generación de reportes de morosidad con actualización de filtros.
RF25	Reporte de pagos realizados	Permite la generación de reportes de pagos recibidos con criterios configurables.

2.4. Requerimientos no funcionales

RNF01	Usabilidad y Experiencia	El sistema debe ser intuitivo, fácil de usar y comprensible para usuarios administrativos y residentes con distintos niveles de alfabetización digital.
RNF02	Usabilidad y Experiencia	La interfaz debe cumplir criterios básicos de accesibilidad visual y de navegación.
RNF03	Usabilidad y Experiencia	El sistema debe ser accesible desde navegadores web modernos y dispositivos móviles.
RNF04	Rendimiento y Capacidad	El sistema debe responder a las operaciones comunes de consulta en un tiempo no mayor

		a 3 segundos bajo carga normal.
RNF05	Rendimiento y Capacidad	Debe permitir el uso simultáneo de múltiples usuarios sin degradación crítica del servicio.

2.5. Supuestos y dependencias

Integridad de Datos: Se asume que la administración entregará la base de datos de los 160 departamentos actualizada para la carga inicial.

Conectividad: La operatividad depende de que el edificio cuente con acceso a internet estable en conserjería y administración.

3. ESTILOS Y PATRONES ARQUITECTÓNICOS

3.1. Estilo arquitectónico adoptado

Para el sistema "Soluciones MSV", se ha adoptado una **Arquitectura de Microservicios** (o arquitectura altamente modular distribuida), operando bajo un modelo general **Cliente-Servidor**. El backend de la aplicación no está construido como un bloque único (monolito), sino que está dividido en servicios independientes, cohesivos y con responsabilidades únicas (Módulo de Usuarios, Residentes, Financiero, Comunicaciones, Solicitudes y Reportes). A nivel de infraestructura, la capa de presentación (Cliente Web y App Móvil) interactúa con estos servicios a través de la red.

3.2. Justificación del estilo según el contexto del sistema

La elección de este estilo arquitectónico responde directamente a las necesidades del edificio "El Mirador" y a la estrategia de desarrollo:

- **Escalabilidad Independiente:** Durante los primeros días del mes, el "Motor Financiero y Cobranza" recibirá un alto volumen de tráfico. Al estar separado de los demás módulos, este servicio puede escalar sus recursos en la nube sin necesidad de replicar todo el sistema.
- **Tolerancia a fallos:** Si el "Módulo de Comunicaciones" (o el servicio externo de notificaciones) sufre una caída temporal, los residentes aún podrán ingresar al portal y realizar sus pagos, ya que el "Motor Financiero" opera de forma autónoma con su propia base de datos.
- **Seguridad y Aislamiento de Datos:** Al separar la información financiera de la información de paquetería o solicitudes comunes, se reduce el riesgo de

accesos no autorizados a datos sensibles.

3.3. Patrones de diseño aplicados

Para dar soporte al estilo arquitectónico descrito, se han implementado los siguientes patrones de diseño de software e infraestructura:

- **Patrón MVC (Modelo-Vista-Controlador):** Implementado principalmente en la interacción general del sistema. Las aplicaciones cliente (Web y Móvil) actúan como la *Vista*, la Puerta de Enlace (API Gateway) enruta las peticiones actuando como *Controlador* inicial, y los Módulos de Backend junto con sus bases de datos conforman el *Modelo* que contiene la lógica de negocio.
- **API Gateway (Puerta de Enlace):** Aplicado en el servidor de aplicaciones para proveer un punto de entrada único a los clientes (dispositivos móviles y PC). Este patrón centraliza el enrutamiento de las peticiones HTTP hacia el microservicio correspondiente, facilitando la gestión de seguridad y el control de tráfico.
- **Database per Service (Base de datos por servicio):** Como se evidencia en la Vista de Implementación, cada módulo de negocio tiene su propio almacenamiento persistente (ej. *Base de Datos de Residentes*, *Base de Datos de Solicitudes*). Esto garantiza un bajo acoplamiento y asegura que los cambios en el esquema de datos de un módulo no afecten a los demás.
- **Patrón CQRS (Command and Query Responsibility Segregation) / Materialized View:** Aplicado específicamente en el **Módulo de Reportes y Análisis**. Este módulo consume datos de una "*Base de Datos de Lectura*", la cual es una copia de la información de otras bases de datos transaccionales.

4. MODELO 4 +1 Y VISTAS ARQUITECTÓNICAS

4.1. VISTA DE ESCENARIO

4.1.1. Propósito

El propósito del Diagrama de caso de uso es visualizar y definir los requerimientos funcionales de "Soluciones MVC". Visualizando un mapa que hace el sistema y quienes de los actores interactúan con el sistema.

4.1.2. Actores

Los Actores Principales que se presentan son los siguientes:

-**Administrador:** Es el actor principal responsable de la gestión administrativa del Edificio. Interactúa de manera intensiva con el sistema.

-**Comite Junta de Vecinos:** Este actor es un grupo de residentes que

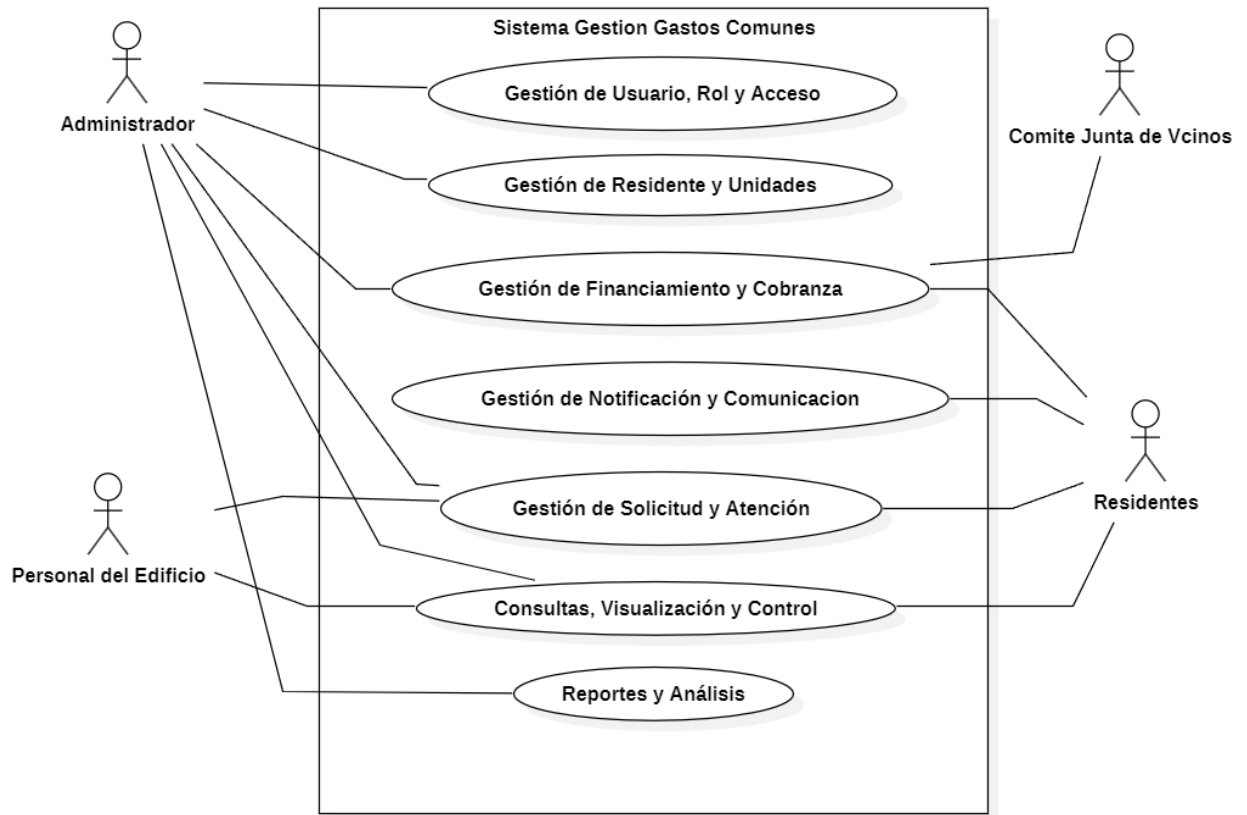
participa en decisiones administrativas.

-Residentes (Propietario/Arrendatario): Este actor utiliza el sistema para consultar información y realizar acciones relacionadas con su unidad habitacional.

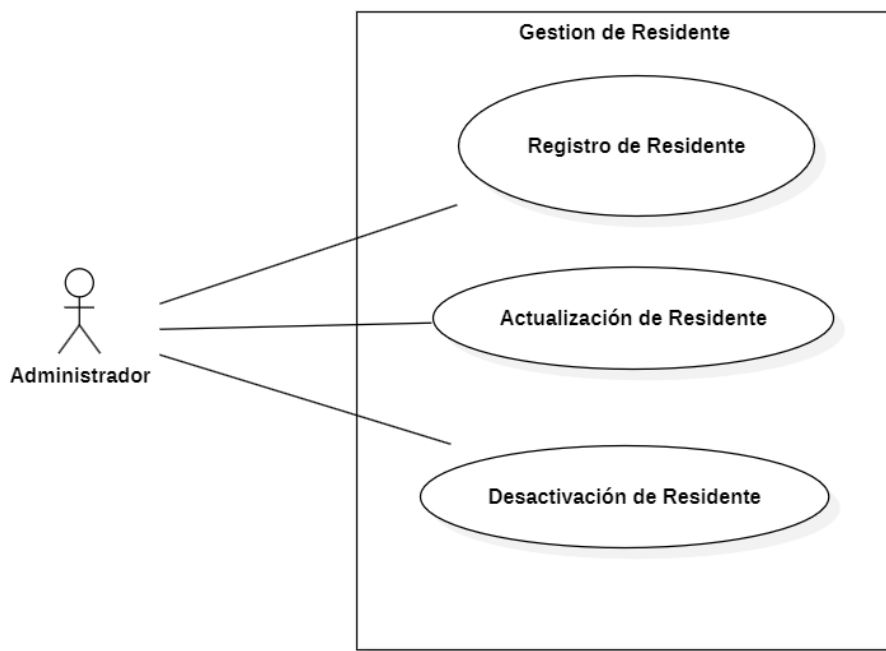
-Personal del Edificio: Este actor Incluye a los conserjes y personal de

mantenimiento.

4.1.3. Diagrama general de casos de uso



4.1.4. Diagrama de casos de uso específicos



4.1.5. Lista de casos de uso

Código	Nombre	Actores
CU-001-001	Exportar saldos y puntos a vencer	
CU-002-001	Exportar actividades	
CU-002-002	Importar deuda vencida por PDV	
CU-004-001	Generación Archivo PDA Importación	

4.1.6. Especificación de casos de uso

Caso de Uso	Gestión de residentes	Identificador: RF-04
Actor(es)	Administrador	
Tipo	Principal	
Referencias	Gestión de Residentes y Unidad	
Precondición	-El Administrador debe haber iniciado sesión -La Unidad Habitacional al que se asignará el residente ya debe estar registrado y Activo	
Postcondición	-Registro/Actualización: los datos del residente quedan guardados o modificados en la base de datos, vinculados a su unidad y con el estado "Activo". -Desactivación: El estado del residente cambia a "Inactivo" así bloqueando sus accesos, pero su registro se conserva intacto para	

	mantener la trazabilidad histórica exigida.
Descripción	El sistema debe permitir el registro, actualización y desactivación de residentes, manteniendo trazabilidad histórica.
Resumen	[Resumen de alto nivel del funcionamiento]

CURSO NORMAL

Nro.	Ejecutor	Paso o Actividad
Paso 1	Administrador	Selecciona la opción "Gestión de Residente" en el menú.
Paso 2	Administrador	Hacer clic en la opción "Registrar Nuevo Residente".
Paso 3	Administrador	Seleccione al Usuario al que desea registrar
Paso 4	Administrador	Defina qué tipo de residente es "Propietario/Arrendatario".
Paso 5	Administrador	Haga clic en el botón "Guardar"
Este flujo describe el curso normal cuando el Administrador registra a un nuevo residente sin ningún inconveniente. En el Paso 1 muestra la pantalla de Administración de residentes con el listado actual. En el Paso 2 despliega el formulario del residente que quiere registrar. En el Paso 3 Valida en tiempo real los datos guardados de los usuarios asignados como residente En el paso 4 Habilita el campo de datos para que después sea asignado a un departamento. Y en el paso 5 Registra al residente en la base de datos, vincula el registro al departamento seleccionado, Crea la marca de tiempo para la trazabilidad histórica y Muestra un mensaje de éxito: <i>"Residente registrado correctamente"</i>.		

CURSO ALTERNATIVO

Nro.	Descripción de acciones alternas
Paso 1	En el listado de residentes, buscar y seleccionar al residente que desea retirar, se desplegará la información detallada del mismo
Paso 2	Hace clic en el botón "Modificar Residente" o "Desactivar", en Modificar tiene la opción de si lo dispondrá en Arriendo (Seguirá activo porque sigue siendo propietario del departamento aunque ya no este viviendo en ella) y en Desactivar mostrará un mensaje que si desea realizar esta acción abusando de que el usuario perderá sus accesos sin importar si es Arrendatario o Propietario pero mantendrá su historial
Paso 3	Confirmar la acción haciendo clic en "Aceptar"
Este Curso corresponde a la necesidad de Desactivar a un residente (por ejemplo, porque se mudó) conservando el registro en la base de datos	

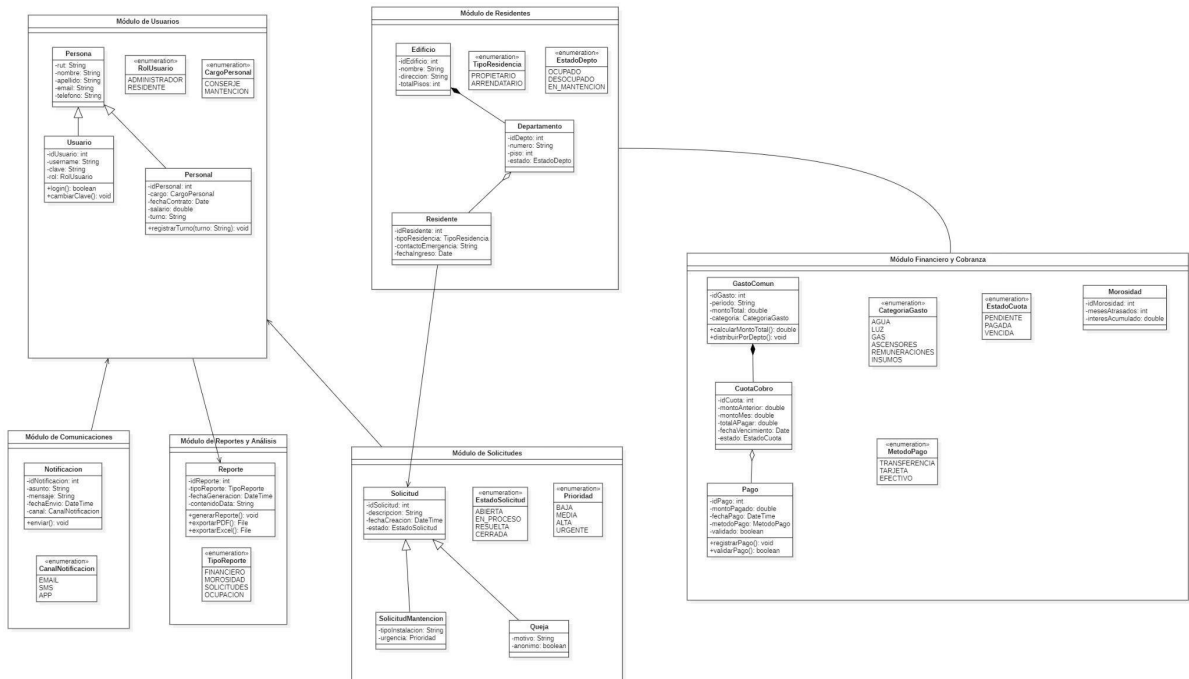
4.2. VISTA LÓGICA

4.2.1. Propósito

El Diagrama de clases es visualizar y definir la estructura estática de

“Soluciones MVC”. Visualizando un mapa de las entidades, sus atributos, métodos y cómo se relacionan jerárquica y operativamente entre sí para organizar la lógica y los datos del sistema.

4.2.2. Diagrama de clases



4.2.3. Descripción diagrama de clases

El diagrama de clases correspondiente al Sistema de Administración del Edificio "El Mirador" está diseñado bajo un enfoque estrictamente **modular y eficiente**, estructurado en 6 paquetes independientes para eliminar la redundancia de información, corregir los problemas de acoplamiento y

asegurar una arquitectura limpia. Su estructura lógica se desarrolla de la siguiente manera:

Módulo de Usuarios: Centraliza la autenticación y los datos personales a través de la clase abstracta **Persona**. A partir de este supertipo, el diagrama se bifurca mediante relaciones de **Herencia (Generalización)** en Usuario (que maneja las credenciales de acceso, el inicio de sesión y los roles mediante la enumeración RolUsuario) y Personal (que añade los atributos laborales del equipo de conserjería y mantenimiento). Esto elimina la necesidad de crear subclases redundantes para cada tipo de trabajador o administrador.

Módulo de Residentes: Gobierna la infraestructura física y el régimen de habitabilidad. La clase Edificio sostiene una relación de **Composición** con Departamento, asegurando la integridad referencial. Para evitar el error de diseño anterior, Residente hereda directamente de Persona y se vincula al Departamento mediante una **Agregación**. El tipo de ocupación (si es dueño o arrendatario) se maneja de forma limpia mediante un atributo basado en la enumeración TipoResidencia, eliminando clases duplicadas.

Módulo Financiero y Cobranza: Diseñado para automatizar procesos y mitigar los problemas de liquidez del edificio. La clase GastoComun consolida los costos globales del periodo y, mediante una relación de **composición**, genera las respectivas CuotaCobro asignadas a cada Departamento. El flujo se completa con la clase Pago, que registra el método y valida la transacción. Para dar respuesta directa al requerimiento del negocio, la clase Morosidad se asocia de forma directa al Departamento, permitiendo identificar instantáneamente los estados de deuda y aplicar intereses sin procesos manuales.

Módulo de Comunicaciones: Actúa como una capa de transparencia hacia la comunidad. Utiliza la clase Notificación para despachar de forma automatizada recordatorios de pago o alertas a través de diferentes canales (definidos en CanalNotificacion), asociándose directamente con la clase Usuario para asegurar la trazabilidad del receptor.

Módulo de Solicitudes: Resuelve de manera óptima los requerimientos y reclamos de los cohabitantes a través de la clase abstracta Solicitud, la cual controla los estados y prioridades. Mediante **Herencia**, se especializa en dos ramas limpias de duplicidad: SolicitudMantencion (que registra el requerimiento técnico y se asocia al Personal asignado) y Queja (diseñada

para la gestión de reclamos institucionales).

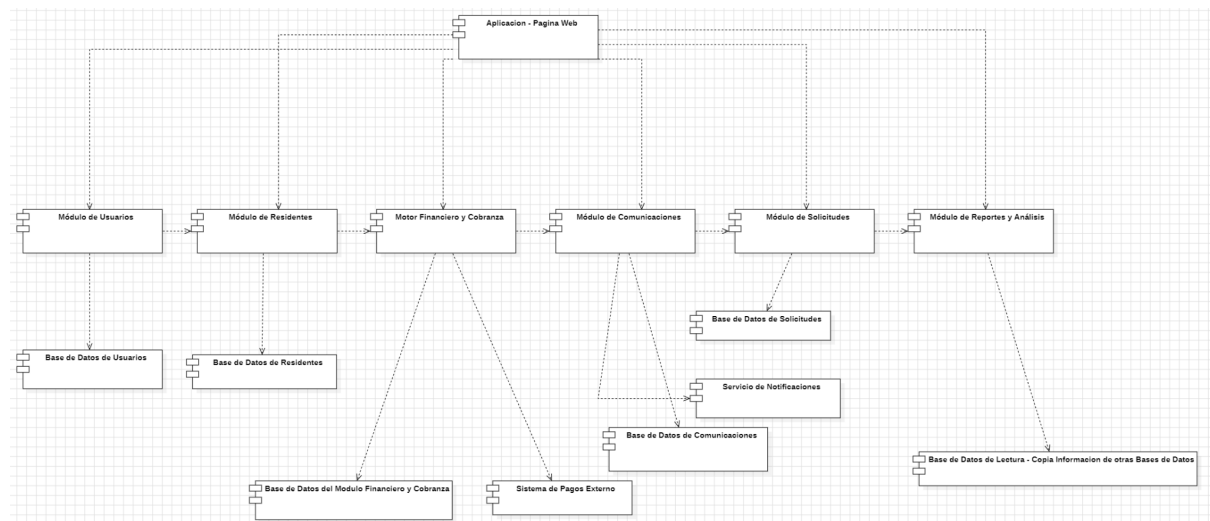
Módulo de Reportes y Análisis: Provee herramientas automatizadas para la toma de decisiones administrativas. A través de la clase Reporte, el sistema procesa la información financiera, de ocupación o de morosidad, permitiendo su exportación nativa a formatos PDF o Excel. Esta entidad se vincula al Usuario que genera la consulta para mantener un estricto control de auditoría.

4.3. VISTA DE IMPLEMENTACIÓN/DESARROLLO

4.3.1. Propósito

El propósito de la Vista de Implementación (o Desarrollo) es ilustrar la organización estática de los módulos de software que componen el sistema "Soluciones MSV". Esta vista permite comprender cómo se estructuran las distintas piezas de código, sus interfaces y las dependencias entre ellas para llevar a cabo la lógica de negocio requerida por el edificio "El Mirador". Para lograr esta representación de manera integral, en esta sección se incluye tanto el **diagrama de componentes**, que detalla los artefactos de software ejecutables y sus conexiones directas, como el **diagrama de paquetes**, el cual agrupa lógicamente estos elementos para facilitar la gestión, el mantenimiento y la escalabilidad del proyecto por parte del equipo de desarrollo.

4.3.2. Diagrama de componente



4.3.3. Descripción diagrama de componente

El diagrama de componentes de "Soluciones MSV" evidencia una arquitectura

modular orientada a la separación de responsabilidades. Se organiza lógicamente en tres niveles principales de interacción:

1. **Capa de Interfaz de Usuario:**

- **Aplicación - Página Web:** Es el componente central de interacción o *frontend*. Actúa como el punto de entrada para todos los actores (Administrador, Residentes, Conserjería). Este componente orquesta las peticiones y depende de todos los módulos de negocio subyacentes para mostrar la información al usuario.

2. **Capa de Lógica de Negocio (Módulos Centrales):**

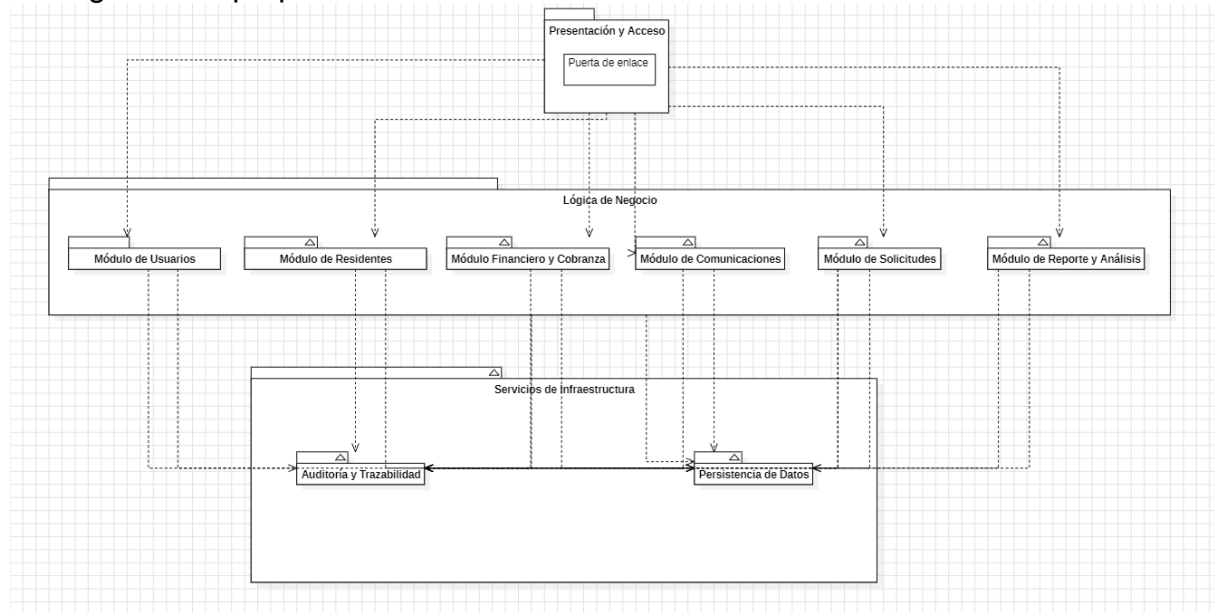
- El sistema se divide en seis componentes principales que encapsulan las reglas del negocio, los cuales tienen dependencias secuenciales entre sí para garantizar el flujo de la información:
 - **Módulo de Usuarios:** Gestiona la autenticación y autorización. Se enlaza con el **Módulo de Residentes**, el cual maneja la información específica de propietarios, arrendatarios y la asignación de los departamentos.
 - **Motor Financiero y Cobranza:** Es el núcleo del sistema de pagos y morosidad. Depende de la información de los residentes para calcular y asignar los Gastos Comunes.
 - **Módulo de Comunicaciones y Módulo de Solicitudes:** Encargados de gestionar la mensajería interna, los recordatorios, el registro de paquetería y la canalización de quejas o tickets de mantención.
 - **Módulo de Reportes y Análisis:** Componente final en la cadena que recopila la información de los módulos anteriores para generar los informes solicitados por la administración.

3. **Capa de Persistencia y Servicios Externos:**

- **Bases de Datos Desacopladas:** Para asegurar una alta cohesión y evitar cuellos de botella, cada módulo principal gestiona su propia base de datos de forma independiente.
- **Base de Datos de Lectura:** El Módulo de Reportes y Análisis consume datos de una base de datos específica de solo lectura (que actúa como réplica o *Data Warehouse*), lo que optimiza el rendimiento al no sobrecargar las bases de datos transaccionales durante la generación de informes pesados.
- **Integraciones Externas:** El diagrama muestra cómo el Motor Financiero y Cobranza se apoya en un **Sistema de Pagos Externo** (como una pasarela bancaria) para procesar las transacciones de los residentes, mientras que el Módulo de Comunicaciones delega el envío

de correos y alertas a un **Servicio de Notificaciones**.

4.3.4. Diagrama de paquete



4.3.5. Descripción diagrama de paquete

El diagrama de paquetes representa la arquitectura lógica del sistema "Soluciones MVC". Su diseño se estructura en tres capas jerárquicas independientes para asegurar una clara separación de responsabilidades y facilitar el mantenimiento del software:

Capa de Presentación y Acceso: Es el punto de entrada único al sistema. Contiene la **Puerta de enlace**, encargada de recibir las solicitudes entrantes, gestionar la seguridad y enrutar las peticiones hacia la capa inferior.

Capa de Lógica de Negocio: Es el núcleo del sistema y agrupa las reglas del negocio del condominio. Recibe las peticiones desde la Puerta de enlace y se subdivide en seis módulos altamente cohesivos:

- **Módulo de Usuarios:** Controla el ciclo de vida de los usuarios del sistema, la asignación de perfiles y los mecanismos de autenticación.
- **Módulo de Residentes:** Administra la información de los habitantes, contratos de propietarios o arrendatarios, y el estado de los departamentos.
- **Módulo Financiero y Cobranza:** Centraliza la lógica compleja de cálculo de gastos comunes, generación de cuotas, procesamiento de pagos y control de morosidades.
- **Módulo de Comunicaciones:** Se encarga de la lógica de distribución

de avisos y alertas dirigidas a los usuarios.

- **Módulo de Solicitudes:** Procesa el flujo, estados y asignación de requerimientos de mantenimiento y quejas.
- **Módulo de Reporte y Análisis:** Consolida la información operativa y financiera para estructurar balances y auditorías.

Capa de Servicios de Infraestructura: Provee soporte técnico transversal a todos los módulos de la capa de negocio. Está compuesto por dos paquetes críticos:

- **Auditoría y Trazabilidad:** Registra de manera independiente el historial de eventos, cambios de estado y acciones de los usuarios para garantizar la seguridad del sistema.
- **Persistencia de Datos:** Abstrae el acceso al motor de la base de datos, encargándose de la persistencia y recuperación de toda la información transaccional.

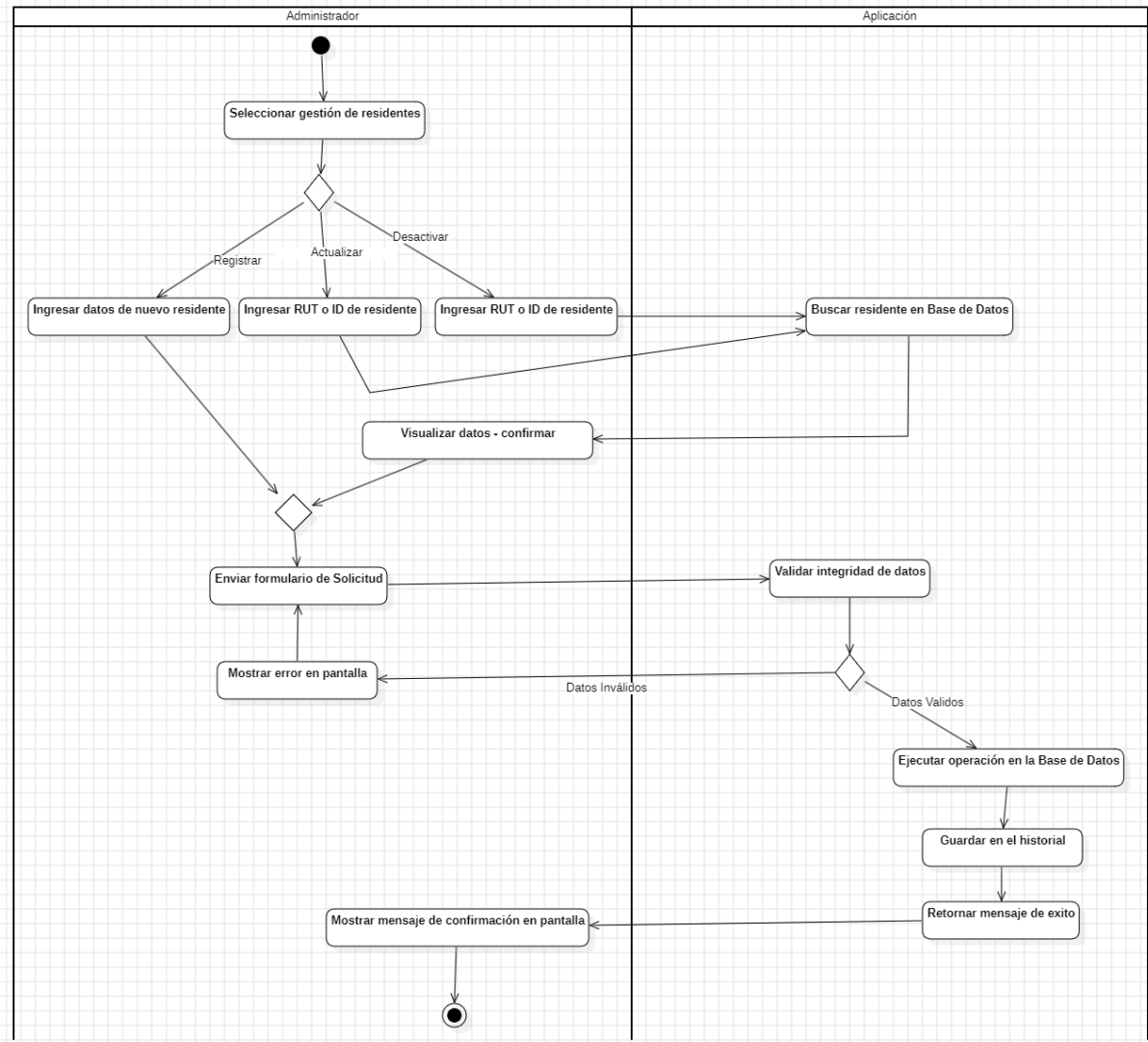
4.4. VISTA DE PROCESOS

4.4.1. Propósito

El propósito de la Vista de Procesos es describir el comportamiento dinámico del sistema "Soluciones MSV", mostrando cómo fluye la información y cómo interactúan los actores con los componentes de la aplicación. Específicamente, este apartado modela el flujo de trabajo del módulo de "Gestión de Residentes", detallando la secuencia de actividades, las decisiones lógicas y las validaciones internas que realiza

el sistema.

4.4.2. Diagrama de actividad



4.4.3. Descripción diagrama de actividad

El diagrama de actividad correspondiente a la **Gestión de Residentes** está estructurado en dos particiones o *swimlanes*: **Administrador** (usuario del sistema) y **Aplicación** (lógica del sistema). El flujo de eventos se desarrolla de la siguiente manera:

1. **Inicio del proceso:** El flujo comienza cuando el Administrador ejecuta la acción principal de "Seleccionar gestión de residentes" en la interfaz.
2. **Bifurcación de la operación (Nodo de decisión):** El sistema presenta tres

caminos posibles dependiendo de la intención del administrador:

- **Registrar:** El administrador procede a "Ingresar datos de nuevo residente". Este camino avanza directamente hacia el paso de envío.
 - **Actualizar o Desactivar:** El administrador debe "Ingresar RUT o ID de residente". Al hacerlo, la Aplicación ejecuta la acción de "Buscar residente en Base de Datos". Tras recuperar la información, el administrador debe "Visualizar datos - confirmar" antes de proseguir.
3. **Envío de la solicitud:** Los caminos anteriores convergen. El administrador procede a "Enviar formulario de Solicitud" hacia el sistema.
 4. **Validación (Nodo de decisión de integridad):** La Aplicación recibe la solicitud y ejecuta la acción "Validar integridad de datos".
 - **Datos Inválidos:** Si la validación falla (ej. campos vacíos o formatos incorrectos), el flujo retorna a la capa del administrador para "Mostrar error en pantalla". El administrador debe corregir los datos y volver a "Enviar formulario de Solicitud" (creando un ciclo de corrección).
 - **Datos Válidos:** Si la validación es exitosa, el flujo continúa su curso dentro de la aplicación.
 5. **Ejecución y Persistencia:** La Aplicación procede a "Ejecutar operación en la Base de Datos" (completando la inserción, modificación o desactivación del residente). Inmediatamente después, ejecuta "Guardar en el historial" para asegurar la trazabilidad y auditoría de los cambios.
 6. **Cierre y Confirmación:** Finalmente, la Aplicación procede a "Retornar mensaje de éxito", lo que detona la acción final en el lado del Administrador: "Mostrar mensaje de confirmación en pantalla". Tras esto, se alcanza el nodo final, terminando el proceso.

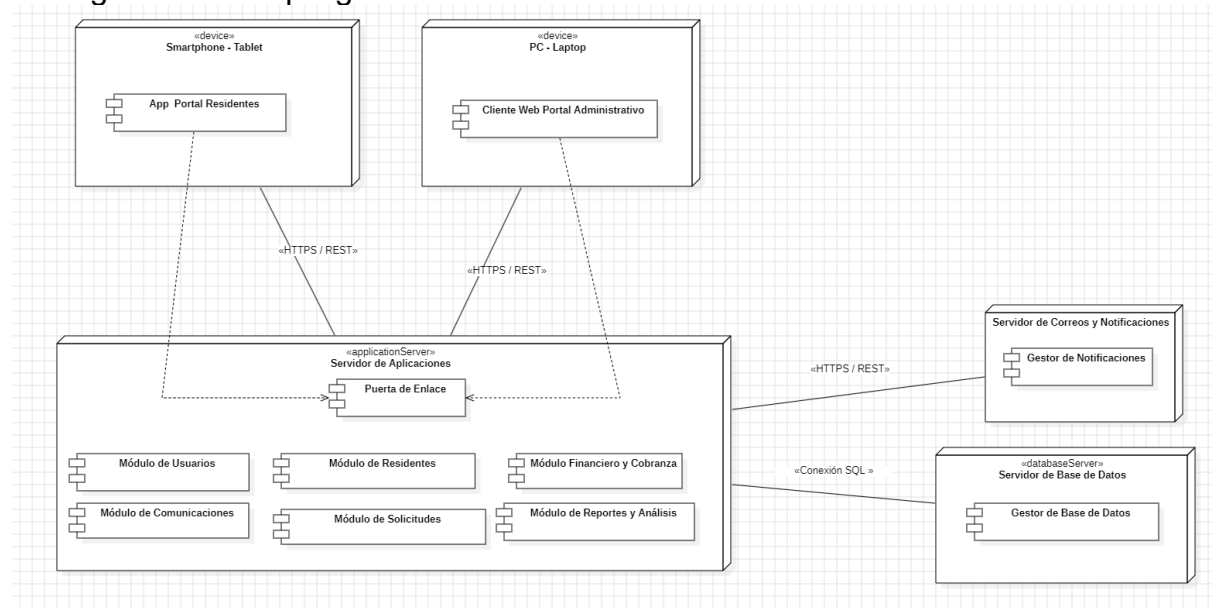
4.5. VISTA FÍSICA

4.5.1. Propósito

El propósito de la Vista Física es ilustrar la topología de hardware y software del sistema "Soluciones MSV", mostrando cómo los distintos artefactos y componentes de software se distribuyen y ejecutan en los nodos físicos (dispositivos y servidores). Esta vista define la infraestructura tecnológica necesaria para soportar las operaciones del edificio "El Mirador", asegurando que se cumplan los atributos de disponibilidad, escalabilidad y seguridad requeridos tanto por la

administración como por los residentes.

4.5.2. Diagrama de despliegue



4.5.3. Descripción diagrama de despliegue

El diagrama de despliegue está estructurado en base a diferentes nodos que se comunican a través de protocolos de red estándar. La distribución es la siguiente:

1. Dispositivos Cliente (Nodos de Acceso):

- **<<device>> Smartphone - Tablet:** Dispositivos móviles utilizados principalmente por los residentes y el personal operativo. Ejecutan el componente **"App Portal Residentes"**.
- **<<device>> PC - Laptop:** Computadores de escritorio o portátiles, utilizados principalmente en conserjería y la oficina de administración. Ejecutan el componente **"Cliente Web Portal Administrativo"**.

2. Servidor de Aplicaciones <<applicationServer>>:

- Es el nodo central (alojado en infraestructura Cloud) que contiene la lógica de

- negocio del sistema.
- Recibe las peticiones externas a través de una **"Puerta de Enlace"** (Gateway), la cual enruta las solicitudes hacia los distintos módulos internos según corresponda: *Módulo de Usuarios, Módulo de Residentes, Módulo Financiero y Cobranza, Módulo de Comunicaciones, Módulo de Solicitudes y Módulo de Reportes y Análisis.*
3. **Servidor de Correos y Notificaciones:**
- Un nodo especializado que aloja el **"Gestor de Notificaciones"**. Su función es descongestionar el servidor principal manejando el envío masivo de correos electrónicos y alertas (ej. avisos de paquetería o recordatorios de morosidad).
4. **Servidor de Base de Datos<<databaseServer>> :**
- Nodo dedicado al almacenamiento persistente, transaccional y seguro de la información del edificio (historial financiero, directorio de residentes, registros de conserjería). Contiene el **"Gestor de Base de Datos"**.
 - *Comunicación:* Se enlaza con el Servidor de Aplicaciones mediante una **<<Conexión SQL>>**, garantizando consultas rápidas y manteniendo la integridad de las bases de datos relacionales necesarias para el módulo financiero.

5. REQUISITOS DE CALIDAD

5.1. Propósito

5.2. Atributos de calidad (por ejemplo: Usabilidad, Accesibilidad (WCAG), Rendimiento, Mantenibilidad, Seguridad Portabilidad)

ATRIBUTO DE CALIDAD	DESCRIPCION	JUSTIFICACIÓN

5.3. Reglas y criterios de evaluación de calidad

Cómo se medirá el cumplimiento de cada atributo (ej. tiempo de carga < 2 seg, puntuación Nielsen de usabilidad, cumplimiento WCAG nivel AA, etc.).

Herramientas o métodos que se utilizarán(pruebas de carga, pruebas heurísticas, inspecciones, validación con usuarios, etc)

6. PRINCIPIOS DE DISEÑO APLICADOS

6.1. Propósito

6.2. Principios de diseño (por ejemplo: abstracción, acoplamiento, cohesión, encapsulamiento, modularidad)

PRINCIPIO	DESCRIPCIÓN	APLICACIÓN EN EL SISTEMA
Cohesión	Cada módulo o clase tiene una	Los servicios están diseñados para

	única responsabilidad bien definida.	realizar tareas específicas y no múltiples funciones

7. PROTOTIPO

7.1. Propósito

7.2. Mockups (imágenes con una breve descripción)

7.3. Justificar herramientas de prototipado

8. EVALUACIÓN DE CALIDAD HEURÍSTICA DE NIELSEN

8.1. Propósito

8.2. Lista de verificación

8.3. Análisis y métricas de resultados

9. CONTROL DE VERSIONES

9.1. Propósito

9.2. Control de versión utilizado (justificar el tipo de control de versión utilizado (fecha, semántica o secuencial)

9.3. Justificar herramientas de versionamiento

7. CONCLUSIONES

8. BIBLIOGRAFÍA

9. ANEXOS

9.1. Carta Gantt